LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

"Transaksi Dinamis (Polymorphism)"

Disusun Oleh:

Sofian Arba'i (2511533029)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

M Dawi Syauqi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

1. Tahapan Praktikum

Berikut adalah beberapa tahapan untuk praktikum Pemrograman Berorientasi Objek pekan 5 dengan judul ”Transaksi Dinamis (Polymorphism)”.

1.1 Tujuan

Tujuan dari praktikum pekan 5 dengan judul ”Transaksi Dinamis (Polymorphism)” sebagai berikut.

1. Memahami konsep Polimorfisme) dan pengikatan dinamis (Dynamic Method Dispatch)
2. Melakukan Method Overriding untuk memodifikasi behavior kelas induk pada kelas anak.
3. Merancang metode (method) yang dapat menerima argumen polimorfik yaitu objek dari berbagai subclass.

1.2 Alat

Alat yang dibutuhkan untuk melakukan praktikum pekan 5 sebagai berikut:

1. Laptop/PC
2. JDK dan Eclipse

1.3 Pemahaman Teori

Kata Polymorphism berasal dari bahasa Yunani yang berarti "memiliki banyak bentuk". Dalam PBO, polimorfisme memungkinkan satu antarmuka menggunakan nama method yang sama untuk dieksekusi dengan cara yang berbeda-beda tergantung pada tipe objek aslinya pada saat runtime / waktu eksekusi.

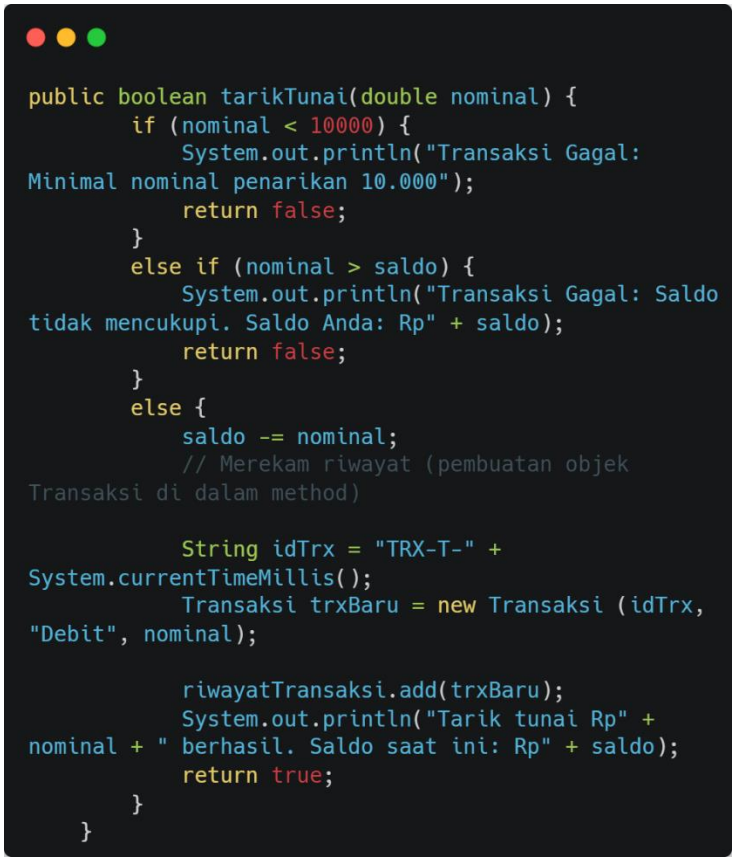
1. Method Overriding (Runtime Polymorphism): Eckel (2006) dalam *Thinking in Java* menjelaskan bahwa overriding terjadi ketika subclass mendefinisikan ulang method milik kelas induknya. Method tersebut harus memiliki nama dan parameter yang persis sama dengan method di kelas induk. Jika objek diinstansiasi dari kelas anak, Java akan selalu mengeksekusi versi method milik kelas anak.
2. Dynamic Method Dispatch (Late Binding): Menurut Oracle (2023), JVM menentukan method mana yang akan dipanggil pada saat program berjalan, bukan pada saat kompilasi. Jika kita memanggil akunAktif.tarikTunai(), JVM akan melihat apakah akunAktif itu berisi Tabungan biasa atau Giro, lalu memanggil logika yang tepat tanpa kita harus menuliskan struktur if-else atau switch-case.
3. Anotasi @Override: Digunakan di Java sebagai praktik terbaik (best practice) untuk memberi tahu compiler bahwa kita berniat menimpa method induk. Jika terjadi kesalahan ketik pada nama method, compiler akan memberikan pesan eror (Sierra dan Bates, 2005).

1.4 Langkah-Langkah

Pada praktikum pekan 4, saya sudah membuat RekeningGiro, tetapi logika tarikTunai-nya masih menggunakan logika Rekening biasa yaitu saldo tidak boleh minus. Pada praktikum ini, saya akan melakukan Overriding agar Giro mengizinkan saldo minus hingga batas overdraft-nya.

1.4.1 Langkah 1: Refaktor Method tarikTunai pada Kelas Induk (Rekening.java)

Membuka file Rekening.java dan mengubah method tarikTunai yang sebelumnya bertipe kembalian void menjadi boolean, agar method tersebut dapat mengembalikan nilai true jika penarikan berhasil dan false jika penarikan gagal. Berikut merupakan perubahan kode program langkah 1.



```
public boolean tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal:
Minimal nominal penarikan 10.000");
            return false;
        }
        else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo
tidak mencukupi. Saldo Anda: Rp" + saldo);
            return false;
        }
        else {
            saldo -= nominal;
            // Merekam riwayat (pembuatan objek
Transaksi di dalam method)

            String idTrx = "TRX-T-" +
System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi (idTrx,
"Debit", nominal);

            riwayatTransaksi.add(trxBaru);
            System.out.println("Tarik tunai Rp" +
nominal + " berhasil. Saldo saat ini: Rp" + saldo);
            return true;
        }
    }
```

Kode Program 1.4.1

Perubahan yang terjadi pada kode program pekan 5 dengan pekan 4 pada langkah 1 yaitu perubahan tipe kembalian method tarikTunai yang sebelumnya void menjadi boolean. Setiap kondisi gagal, yaitu nominal di bawah 10.000 dan nominal yang melebihi saldo, kini mengembalikan false setelah mencetak pesan kesalahan. Sementara itu, penarikan yang berhasil mengurangi saldo, mencatat objek Transaksi berjenis “Debit” dengan kode berawalan “TRX-T-” ke dalam riwayatTransaksi, lalu mengembalikan true. Perubahan tersebut memungkinkan method lain mengetahui hasil penarikan. Pada langkah berikutnya, method transfer dapat memeriksa nilai kembalian tarikTunai dan hanya memanggil setorTunai pada rekening tujuan jika penarikan berhasil. Selain itu, tipe kembalian dan parameter yang sama persis dengan method di kelas induk menjadi syarat agar RekeningGiro dapat menimpa (override) method ini pada langkah 2.

1.4.2 Langkah 2: Melakukan Overriding pada RekeningGiro.java

Membuka file RekeningGiro.java dan menambahkan method tarikTunai yang menimpa (override) method milik kelas induk Rekening, dengan menggunakan anotasi @Override. Method hasil overriding ini memakai aturan batasOverdraft, sehingga rekening giro boleh memiliki saldo negatif selama penarikan tidak melebihi saldo ditambah batas overdraft. Berikut merupakan perubahan kode program langkah 2.

```
package pekan5;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama,
double saldoAwal, String pinAwal, double
batasOverdraft) {
        // Memanggil Inisialisasi dasar dari
superclass
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // IPLEMENTASI POLIMORFISME (OVERRIDING)
    @Override
    public boolean tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal:
Minimal nominal penarikan 10.000");
            return false;
        }
        else if (nominal > (saldo + batasOverdraft)) {
            System.out.println("Transaksi Gagal:
Melebihi batas overdraft! Maksimal penarikan: Rp" +
(saldo + batasOverdraft));
            return false;
        }
        else {
            saldo -= nominal;
            // Merekam riwayat
            String idTrx = "TRX-T-" +
System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx,
"Debit", nominal);

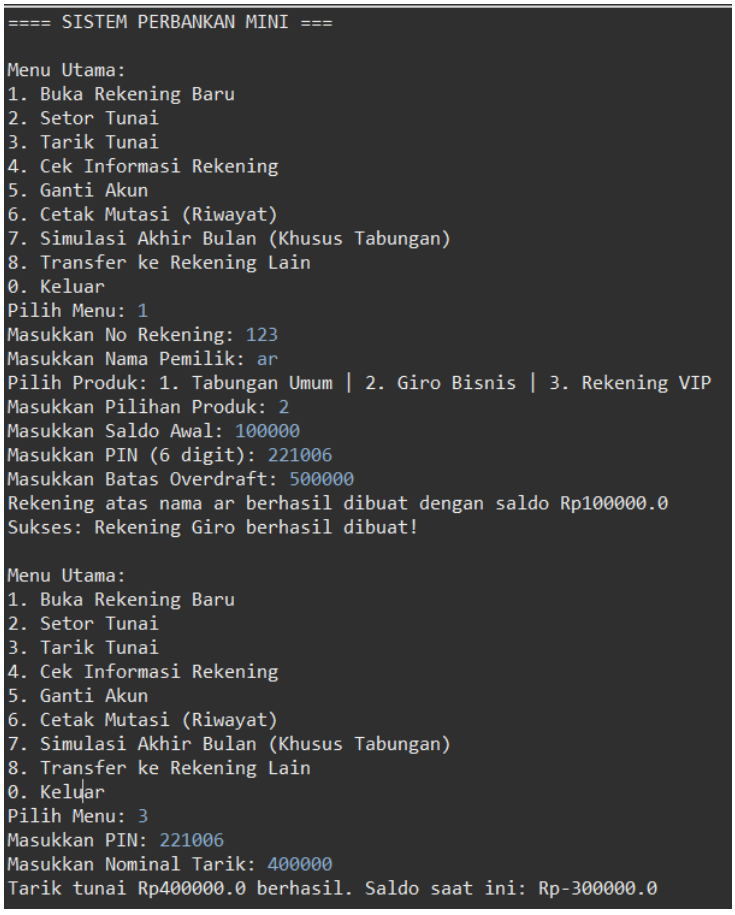
            riwayatTransaksi.add(trxBaru);
            System.out.println("Tarik tunai Rp" +
nominal + " berhasil. Saldo saat ini: Rp" + saldo);
            return true;
        }
    }
    public double getBatasOverdraft(){
        return batasOverdraft;
    }
}
```

Kode Program 1.4.2

Perubahan yang terjadi pada kode program pekan 5 dengan pekan 4 pada langkah 2 yaitu penambahan method tarikTunai pada RekeningGiro yang menimpa method dengan nama, parameter, dan tipe kembalian yang sama pada kelas induk Rekening. Anotasi @Override membuat compiler memeriksa bahwa method tersebut benar-benar menimpa method induk, sehingga kesalahan penulisan nama atau parameter akan terdeteksi sebagai error. Pada versi Rekening, penarikan ditolak jika nominal lebih besar dari saldo. Pada versi RekeningGiro, kondisi tersebut diganti menjadi nominal > (saldo + batasOverdraft), sehingga penarikan masih diizinkan selama total tidak melewati batas overdraft dan saldo dapat bernilai negatif.

1.4.3 Langkah 3: Uji Coba Pengikatan Dinamis

Menjalankan file Main.java tanpa mengubah satu baris pun kode pada menu 3 (Tarik Tunai). Pengujian dilakukan dengan membuat rekening baru menggunakan pilihan produk 2 (Giro Bisnis) dengan saldo awal Rp100.000 dan batas overdraft Rp500.000, kemudian melakukan tarik tunai sebesar Rp400.000. Jika objek tersebut merupakan Rekening biasa, penarikan pasti ditolak karena nominal melebihi saldo. Berikut merupakan hasil pengujian langkah 3.



Output Langkah 3

Hasil pengujian pada langkah 3 menunjukkan bahwa penarikan Rp400.000 berhasil dan saldo berubah menjadi Rp-300.000. Padahal menu 3 pada class Main tetap memanggil akunAktif.tarikTunai(tarik) dengan variabel bertipe Rekening. Hal ini terjadi karena objek yang disimpan di akunAktif sebenarnya adalah RekeningGiro, sehingga JVM memilih method tarikTunai milik RekeningGiro pada saat program berjalan, bukan method milik Rekening. Pemilihan method berdasarkan tipe objek asli pada saat runtime inilah yang disebut dynamic method dispatch (late binding).

2. Latihan/Tugas

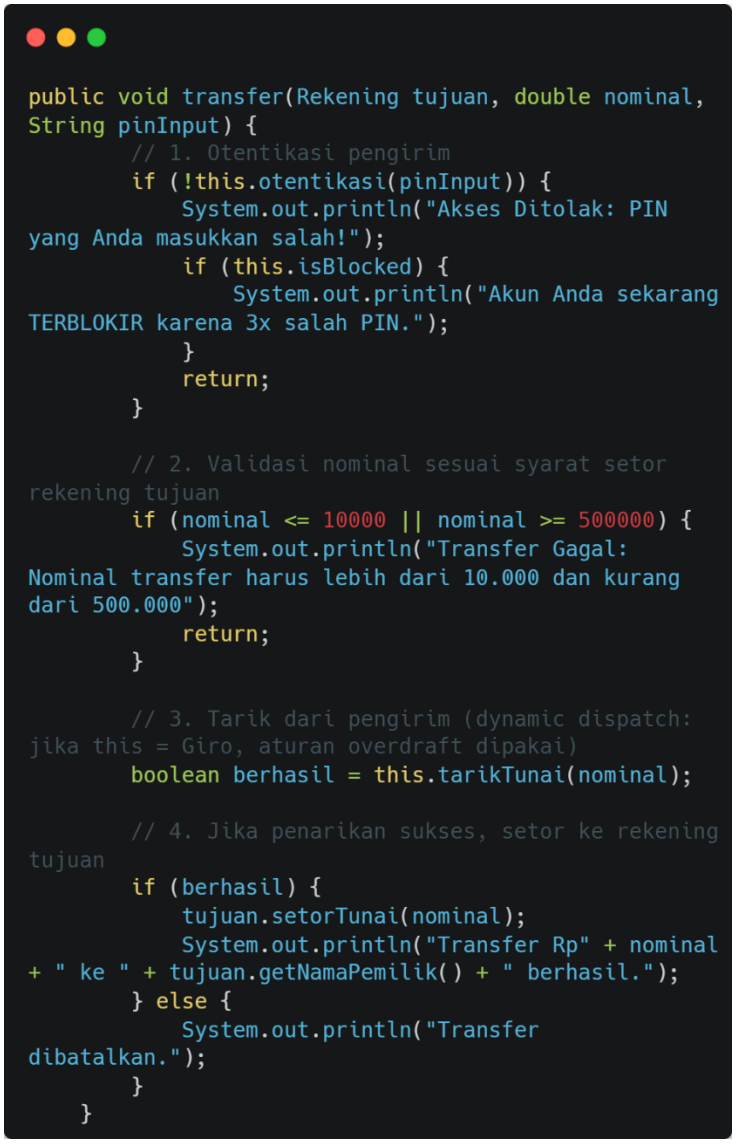
Keindahan sesungguhnya dari Polimorfisme terlihat saat kita melewatkan sebuah objek sebagai parameter ke dalam fungsi, di mana sistem tidak perlu tahu tipe persis objek tersebut, apakah Tabungan atau Giro.

1. Tugas 1: Buat Method Transfer

   Di dalam class Rekening.java, tambahkan satu method bar dengan nama: public void transfer(Rekening tujuan, double nominal, String pinInput). Berikut beberapa ketentuan tugas 1.

   - Method ini menerima parameter polimorfik Rekening tujuan, bisa menerima Tabungan atau Giro.
   - Logika Method. Pertama, lakukan otentikasi(pinInput). Jika false, batalkan, Kedua, panggil this.tarikTunai(nominal). (overriding, jika this adalah Giro, ia otomatis menggunakan aturan overdraft), Ketiga, periksa kembalian nilai tarikTunai. Jika true, panggil tujuan.setorTunai(nominal).

Pada tugas 1, diminta menambahkan method baru bernama transfer pada class Rekening.java dengan parameter Rekening tujuan, double nominal, dan String pinInput. Parameter tujuan bertipe Rekening, sehingga method ini dapat menerima objek RekeningTabungan, RekeningGiro, maupun RekeningVIP sebagai rekening tujuan (parameter polimorfik). Berikut merupakan kode program method transfer.



```
public void transfer(Rekening tujuan, double nominal,
String pinInput) {
        // 1. Otentikasi pengirim
        if (!this.otentikasi(pinInput)) {
            System.out.println("Akses Ditolak: PIN
yang Anda masukkan salah!");
            if (this.isBlocked) {
                System.out.println("Akun Anda sekarang
TERBLOKIR karena 3x salah PIN.");
            }
            return;
        }

        // 2. Validasi nominal sesuai syarat setor
rekening tujuan
        if (nominal <= 10000 || nominal >= 500000) {
            System.out.println("Transfer Gagal:
Nominal transfer harus lebih dari 10.000 dan kurang
dari 500.000");
            return;
        }

        // 3. Tarik dari pengirim (dynamic dispatch:
jika this = Giro, aturan overdraft dipakai)
        boolean berhasil = this.tarikTunai(nominal);

        // 4. Jika penarikan sukses, setor ke rekening
tujuan
        if (berhasil) {
            tujuan.setorTunai(nominal);
            System.out.println("Transfer Rp" + nominal
+ " ke " + tujuan.getNamaPemilik() + " berhasil.");
        } else {
            System.out.println("Transfer
dibatalkan.");
        }
    }
```

Kode Program Tugas 1

Penjelasan kode program method transfer dibagi menjadi beberapa bagian sebagai berikut.

1. Otentikasi Pengirim

   Method memanggil this.otentikasi(pinInput) untuk memeriksa PIN rekening pengirim. Jika PIN salah, program mencetak “Akses Ditolak: PIN yang Anda masukkan salah!” lalu transfer dibatalkan dengan return. Jika kesalahan PIN sudah mencapai 3 kali, program juga mencetak pesan bahwa akun terblokir.

2. Validasi Nominal

   Nominal transfer harus lebih dari 10.000 dan kurang dari 500.000, sesuai syarat method setorTunai milik rekening tujuan. Pengecekan dilakukan sebelum penarikan agar saldo pengirim tidak terpotong ketika setorTunai menolak nominal tersebut.

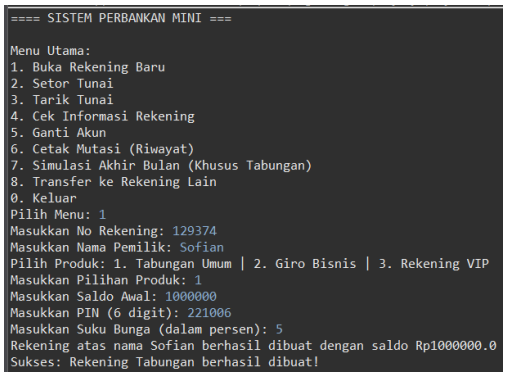
3. Penarikan dari pengirim

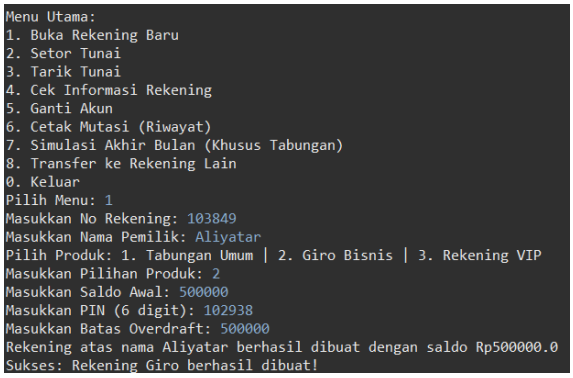
   Method memanggil this.tarikTunai(nominal) dan menyimpan hasilnya pada variabel berhasil. Karena method ini dapat dioverride, jika this adalah RekeningGiro maka versi tarikTunai milik RekeningGiro yang dijalankan sehingga aturan overdraft otomatis berlaku. Inilah penerapan dynamic method dispatch di dalam method transfer.

4. Penyetoran ke Rekening Tujuan

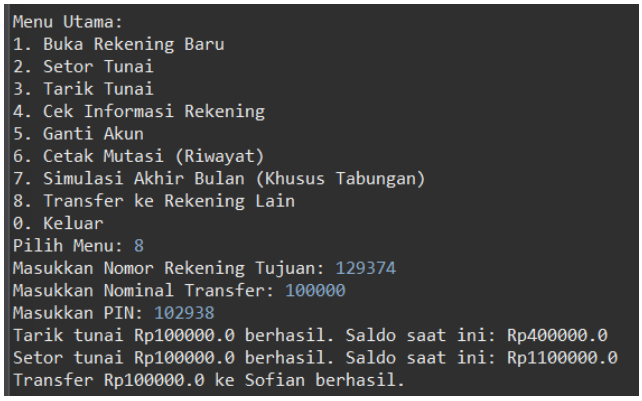
   Jika berhasil bernilai true, method memanggil tujuan.setorTunai(nominal) dan mencetak pesan transfer berhasil beserta nama pemilik rekening tujuan. Jika berhasil bernilai false, program mencetak "Transfer dibatalkan." dan tidak ada saldo yang berpindah.

Output dari tugas 1 sebagai berikut.





Transfer dari rekening aliyatar ke Rekening Sofian

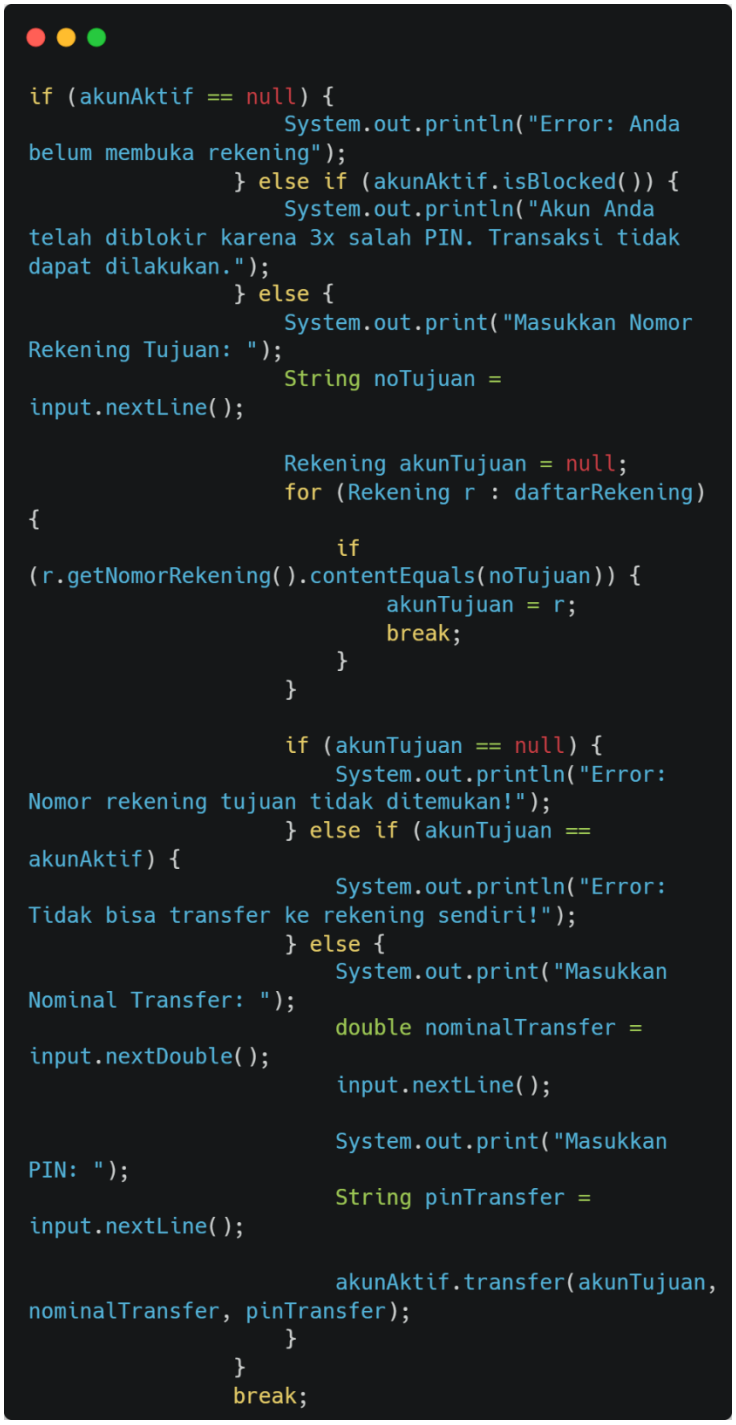


2. Tugas 2: Integrasi ke CLI Utama

Buka Main.java dan tambahkan menu 8. Transfer ke Rekening Lain. Berikut ketentuan dari tugas 2.

- Minta pengguna memasukkan Nomor Rekening Tujuan.
- Cari nomor tersebut di dalam daftarRekening.
- Jika ketemu, minta nominal transfer akunAktif.transfer lalu panggil akunAktif.transfer(akunTujuan, nominal, pin).

Pada tugas 2, diminta membuka Main.java dan menambahkan menu 8 “Transfer ke Rekening Lain”. Pada menu ini, pengguna memasukkan nomor rekening tujuan, program mencari nomor tersebut di dalam daftarRekening, dan jika ditemukan, pengguna diminta memasukkan nominal transfer serta PIN, lalu program memanggil method transfer milik akunAktif. Berikut merupakan kode program untuk menu 8.



```
if (akunAktif == null) {
                    System.out.println("Error: Anda
belum membuka rekening");
                } else if (akunAktif.isBlocked()) {
                    System.out.println("Akun Anda
telah diblokir karena 3x salah PIN. Transaksi tidak
dapat dilakukan.");
                } else {
                    System.out.print("Masukkan Nomor
Rekening Tujuan: ");
                    String noTujuan =
input.nextLine();

                    Rekening akunTujuan = null;
                    for (Rekening r : daftarRekening)
{
                        if
(r.getNomorRekening().contentEquals(noTujuan)) {
                            akunTujuan = r;
                            break;
                        }
                    }

                    if (akunTujuan == null) {
                        System.out.println("Error:
Nomor rekening tujuan tidak ditemukan!");
                    } else if (akunTujuan ==
akunAktif) {
                        System.out.println("Error:
Tidak bisa transfer ke rekening sendiri!");
                    } else {
                        System.out.print("Masukkan
Nominal Transfer: ");
                        double nominalTransfer =
input.nextDouble();
                        input.nextLine();

                        System.out.print("Masukkan
PIN: ");
                        String pinTransfer =
input.nextLine();

                        akunAktif.transfer(akunTujuan,
nominalTransfer, pinTransfer);
                    }
                }
                break;
```

Kode Program Tugas 2

Penjelasan kode program menu 8 dibagi menjadi beberapa bagian sebagai berikut.

1. Pemeriksaan akun aktif dan status blokir

Program memeriksa apakah akunAktif bernilai null. Jika ya, program mencetak

"Error: Anda belum membuka rekening". Jika akun aktif sudah terblokir karena 3x salah PIN, transfer tidak dapat dilakukan dan program mencetak pesan pemblokiran.

2. Pencarian rekening tujuan

   Program meminta nomor rekening tujuan, lalu menelusuri daftarRekening menggunakan perulangan for-each dengan membandingkan getNomorRekening() dari setiap objek. Jika ditemukan, objeknya disimpan pada variabel akunTujuan bertipe Rekening, dan perulangan dihentikan dengan break. Karena bertipe Rekening, variabel ini dapat menampung objek RekeningTabungan, RekeningGiro, maupun RekeningVIP.

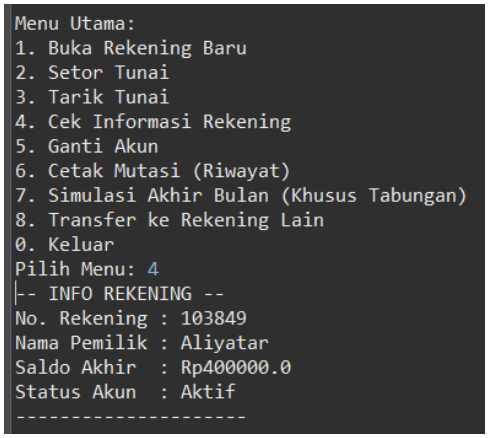
3. Validasi rekening tujuan

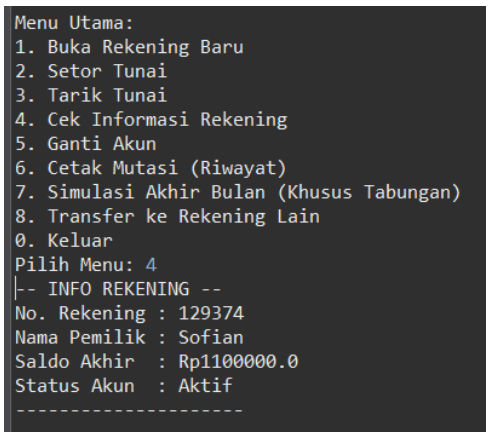
   Jika akunTujuan masih null, program mencetak "Error: Nomor rekening tujuan tidak ditemukan!". Jika akunTujuan sama dengan akunAktif, program mencetak "Error: Tidak bisa transfer ke rekening sendiri!".

4. Input dan pemanggilan transfer

   Jika rekening tujuan valid, program meminta nominal transfer dan PIN, lalu memanggil akunAktif.transfer(akunTujuan, nominalTransfer, pinTransfer). Pemeriksaan PIN, validasi nominal, penarikan, dan penyetoran dilakukan di dalam method transfer.

Output dari tugas 2 sebagai berikut.





3. Link Repository

Berikut dibawah ini merupakan link github untuk laporan praktikum pekan 5. sofianarbai/PrakPBO_C_2511533029 at Praktikum-PBO-Pekan-5